In [2]:
```
import pandas as pd
import psycopg2
from sqlalchemy import create_engine


df = pd.read_csv('creditcard-2.csv')
print(df.shape)
print(df['Class'].value_counts())
```

```
(284807, 31)
Class
0    284315
1       492
Name: count, dtype: int64
```

In [3]:
```
df.head(10)
```

Out[3]:

	Time	V1	V2	V3	V4	V5	V6
0	0.0	-1.359807	-0.072781	2.536347	1.378155	-0.338321	0.462388
1	0.0	1.191857	0.266151	0.166480	0.448154	0.060018	-0.082361
2	1.0	-1.358354	-1.340163	1.773209	0.379780	-0.503198	1.800499
3	1.0	-0.966272	-0.185226	1.792993	-0.863291	-0.010309	1.247203
4	2.0	-1.158233	0.877737	1.548718	0.403034	-0.407193	0.095921
5	2.0	-0.425966	0.960523	1.141109	-0.168252	0.420987	-0.029728
6	4.0	1.229658	0.141004	0.045371	1.202613	0.191881	0.272708
7	7.0	-0.644269	1.417964	1.074380	-0.492199	0.948934	0.428118
8	7.0	-0.894286	0.286157	-0.113192	-0.271526	2.669599	3.721818
9	9.0	-0.338262	1.119593	1.044367	-0.222187	0.499361	-0.246761

10 rows × 31 columns

In [4]:
```
df. describe()
```

Out[4]:

	Time	V1	V2	V3	
count	284807.000000	2.848070e+05	2.848070e+05	2.848070e+05	2.8480
mean	94813.859575	1.168375e-15	3.416908e-16	-1.379537e-15	2.0740
std	47488.145955	1.958696e+00	1.651309e+00	1.516255e+00	1.4158
min	0.000000	-5.640751e+01	-7.271573e+01	-4.832559e+01	-5.6831
25%	54201.500000	-9.203734e-01	-5.985499e-01	-8.903648e-01	-8.4864
50%	84692.000000	1.810880e-02	6.548556e-02	1.798463e-01	-1.98
75%	139320.500000	1.315642e+00	8.037239e-01	1.027196e+00	7.4334
max	172792.000000	2.454930e+00	2.205773e+01	9.382558e+00	1.6875

8 rows × 31 columns

In [5]:
```
# Basic cleaning: Scale Amount/Time (for model)
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
df['Amount_scaled'] = scaler.fit_transform(df[['Amount']])
df['Time_scaled'] = scaler.fit_transform(df[['Time']])
df.drop(['Amount', 'Time'], axis=1, inplace=True)
```

In [6]:
```
# Creating connection to PostgreSQL
engine = create_engine('postgresql+psycopg2://postgres:ernest@local

# Loading data to table
df.to_sql('transactions', engine, if_exists='replace', index=False)
print("Data loaded to PostgreSQL!")
```

```
Data loaded to PostgreSQL!
```

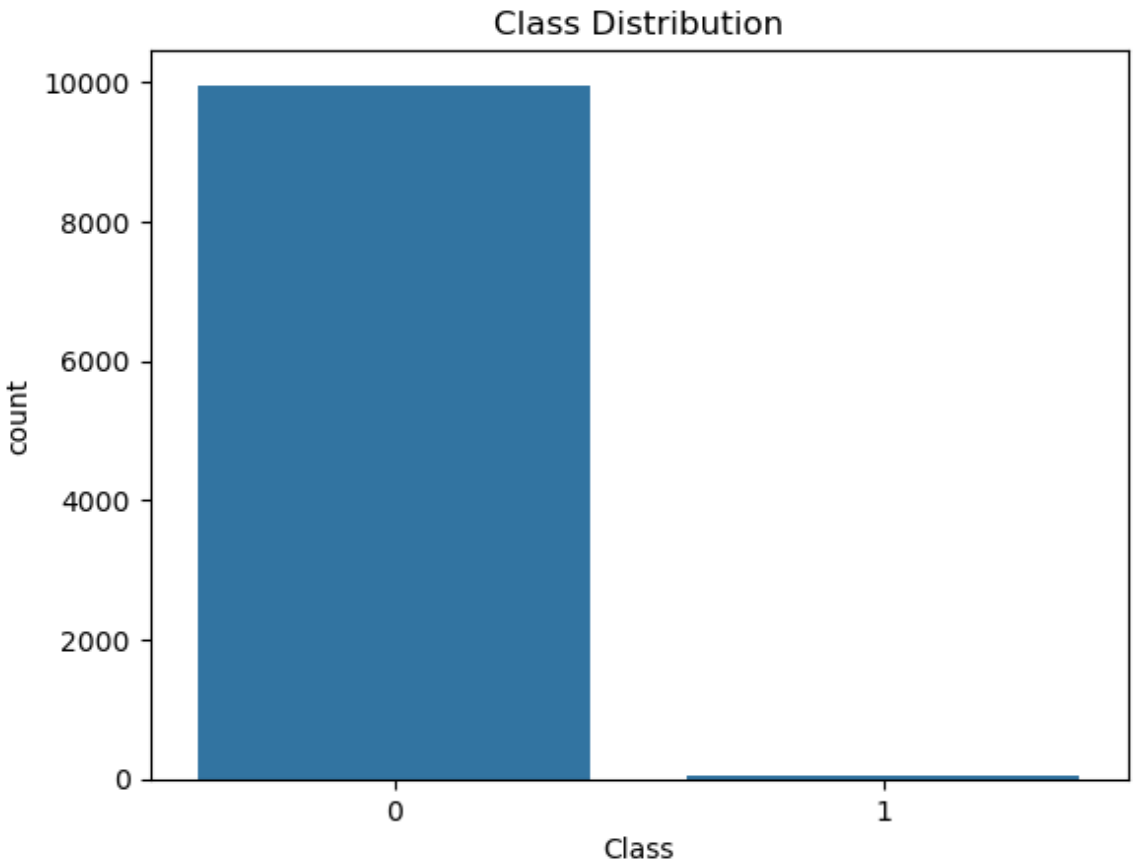
In [7]:
```
# Testing Query from DB
query = "SELECT * FROM transactions LIMIT 10000;"  # Sample for spe
df_sample = pd.read_sql(query, engine)

# Stats
print(df_sample.describe())
```

```
                 V1            V2            V3            V4
V5  \
count  10000.000000  10000.000000  10000.000000  10000.000000  1000
0.000000
mean      -0.241862      0.281949      0.906270      0.264148     -
0.046398
std        1.521679      1.308139      1.159154      1.441235
1.182935
min      -27.670569    -34.607649    -15.496222     -4.657545    -3
2.092129
25%       -1.013283     -0.208342      0.412799     -0.614424     -
```

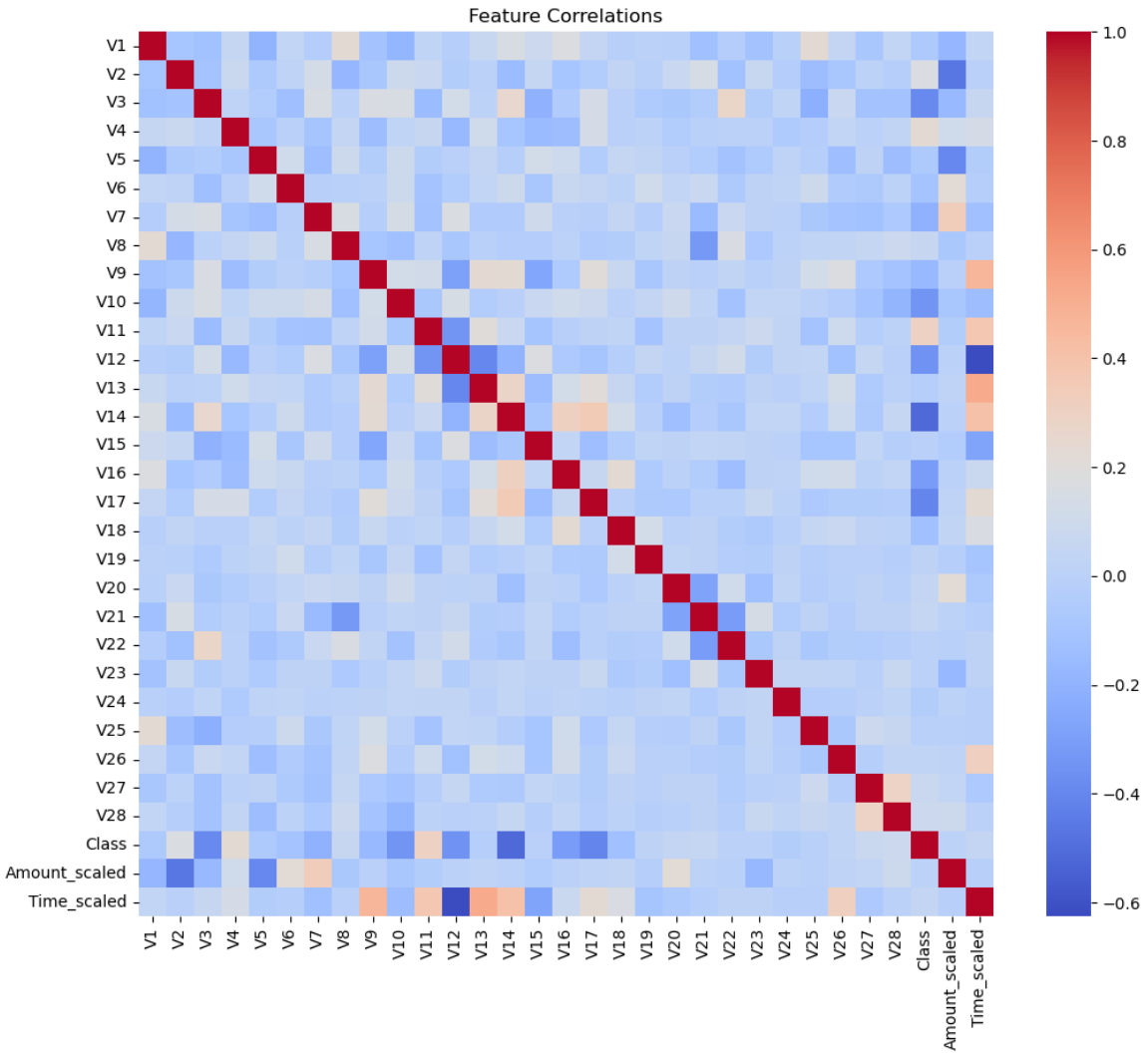
```
0.643390
50%       -0.372799      0.288524      0.944361      0.219861     -
0.152769
75%        1.150864      0.901879      1.602903      1.125666
0.371081
max        1.960497      8.636214      4.101716     10.463020     3
4.099309

                 V6            V7            V8            V9
V10  \
count  10000.000000  10000.000000  10000.000000  10000.000000  1000
0.000000
mean       0.133108     -0.071689     -0.064778      0.802224     -
0.222998
std        1.307311      1.077430      1.259064      1.155198
1.093548
min      -23.496714    -26.548144    -23.632502     -6.329801    -1
3.193415
25%       -0.629934     -0.542336     -0.190747      0.070868     -
0.688422
50%       -0.152566     -0.055585      0.012865      0.805275     -
0.340720
75%        0.505357      0.476280      0.274533      1.506299
0.174295
max       21.393069     34.303177      5.060381     10.392889     1
2.259949

       ...           V22           V23           V24           V25
\
count  ...  10000.000000  10000.000000  10000.000000  10000.000000
mean   ...     -0.152671     -0.033268      0.021335      0.087146
std    ...      0.631083      0.487814      0.594430      0.428171
min    ...     -8.527145    -15.144340     -2.512377     -2.577363
25%    ...     -0.549638     -0.174120     -0.327817     -0.158137
50%    ...     -0.136746     -0.045794      0.079976      0.121001
75%    ...      0.247490      0.081665      0.410877      0.359058
max    ...      4.534454     13.876221      3.200201      5.525093

                V26           V27           V28        Class  Amount
_scaled  \
count  10000.000000  10000.000000  10000.000000  10000.00000   1000
0.000000
mean       0.108140      0.005518      0.002915      0.00380      -
0.101229
std        0.562793      0.410868      0.266247      0.06153
0.737592
min       -1.338556     -7.976100     -3.509250      0.00000      -
0.353229
25%       -0.327974     -0.084489     -0.015753      0.00000      -
0.333239
50%        0.042865     -0.004568      0.015897      0.00000      -
0.289460
75%        0.476394      0.120811      0.077182      0.00000      -
0.149487
max        3.517346      8.254376      4.860769      1.00000      3
0.481731
```

```
        Time_scaled
count  10000.000000
mean      -1.870951
std        0.094201
min       -1.996583
25%       -1.952935
50%       -1.900485
75%       -1.781092
max       -1.680461

[8 rows x 31 columns]
```

In [8]:
```
# Class distribution Plot to check Balance & Imbalance class
import seaborn as sns
import matplotlib.pyplot as plt
sns.countplot(x='Class', data=df_sample)
plt.title('Class Distribution')
plt.show()
```



In [9]:
```
# Correlations (focus on V features)
corr = df_sample.corr()
plt.figure(figsize=(12,10))
sns.heatmap(corr, cmap='coolwarm')
plt.title('Feature Correlations')
plt.show()
```

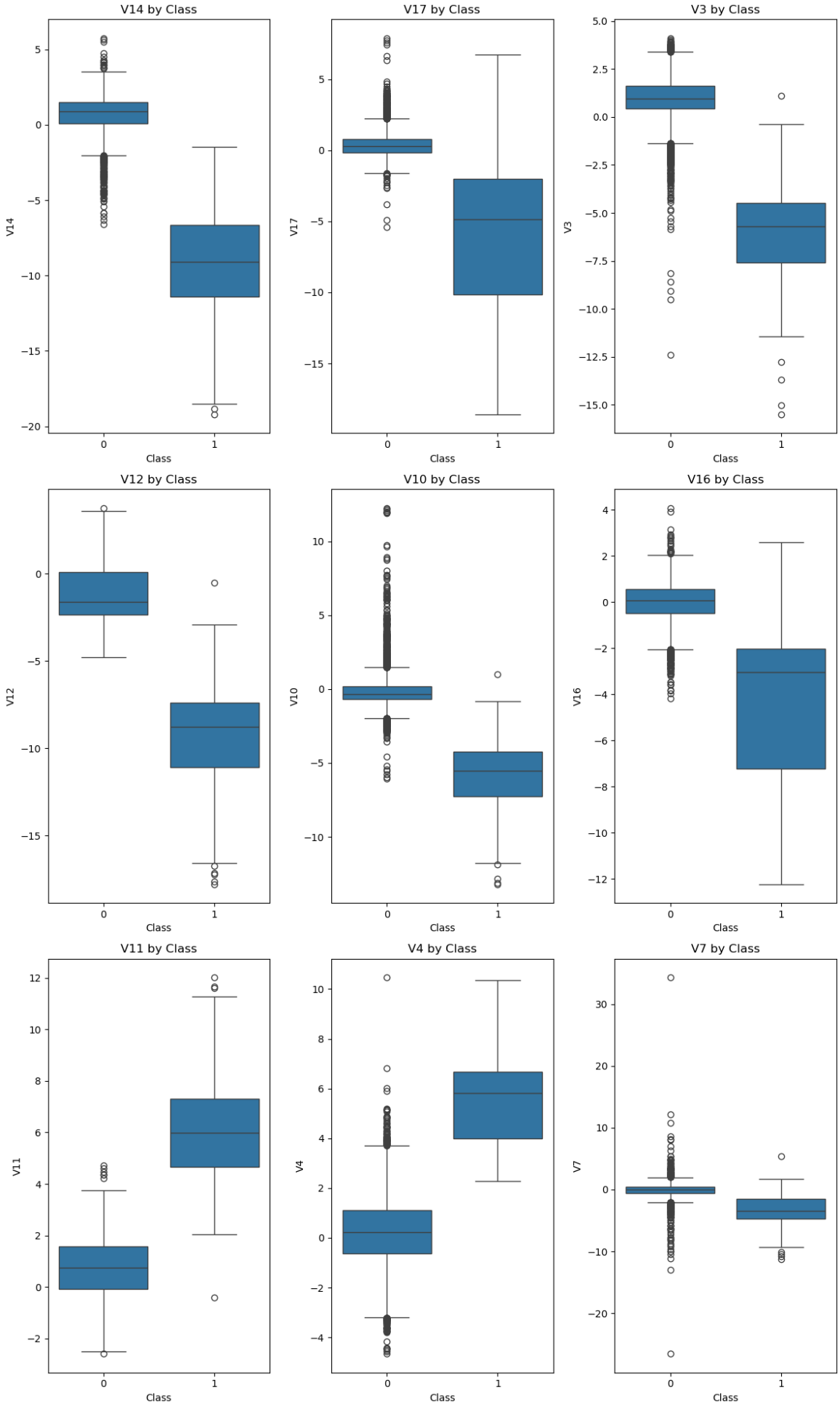


In [10]:
```
# Top 10 features that have the strongest relationship with the tar
target_corr = df_sample.corr()['Class'].drop('Class')
top_features = target_corr.abs().sort_values(ascending=False)
print(top_features.head(10))
```

```
V14    0.517690
V17    0.407361
V3     0.390122
V12    0.351070
V10    0.344365
V16    0.310866
V11    0.297231
V4     0.229448
V7     0.212425
V2     0.166618
Name: Class, dtype: float64
```

In [11]:
```
#correlations with Class
target_corr = df_sample.corr()['Class'].drop('Class')
top_features = target_corr.abs().sort_values(ascending=False).head(

#subplots
fig, axes = plt.subplots(nrows=3, ncols=3, figsize=(12, 20))
axes = axes.flatten()

for i, col in enumerate(top_features):
```

```
    sns.boxplot(x='Class', y=col, data=df_sample, ax=axes[i])
    axes[i].set_title(f'{col} by Class')

plt.tight_layout()
plt.show()
```



In [12]:
```
# Data Preprocessing and Modeling
```

```
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, roc_auc_score
from imblearn.over_sampling import SMOTE


df = pd.read_sql("SELECT * FROM transactions;", engine)
X = df.drop('Class', axis=1)
y = df['Class']
```

In [13]:
```
# Split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size
```

In [14]:
```
# SMOTE approach to generates synthetic samples for the minority cl
# To prevents the model from being biased toward the majority class
smote = SMOTE(random_state=42)
X_res, y_res = smote.fit_resample(X_train, y_train)
```

In [15]:
```
# RandomForest Model
model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_res, y_res)
preds = model.predict(X_test)
print(classification_report(y_test, preds))
print(f'AUC: {roc_auc_score(y_test, preds)}')
```

```
              precision    recall  f1-score   support

           0       1.00      1.00      1.00     56873
           1       0.85      0.83      0.84        89

    accuracy                           1.00     56962
   macro avg       0.93      0.92      0.92     56962
weighted avg       1.00      1.00      1.00     56962

AUC: 0.9156160473453864
```

In [16]:
```
preds = model.predict(X_test)
preds
```

Out[16]:
```
array([0, 0, 0, ..., 0, 0, 0])
```

In [17]:
```
# Save model
import joblib
joblib.dump(model, 'fraud_model.pkl')
```

Out[17]:
```
['fraud_model.pkl']
```

In [18]:
```
df.to_csv('fraud_data_for_tableau.csv', index=False)
```

In [ ]: